PROJECT REPORT

ON

"CampusConnect: Student Academic & Communication Portal"

Submitted in partial fulfillment of the requirement for the award of Degree of

Bachelor of Technology in Computer Science & Engineering

Submitted to:

Rajasthan Technical University, Kota (Raj.)

Submitted By:

Aditya Gour (24EARCS008)

Anjali Kumari (24EARCS020)

Aayushman Pathak (24EARCS003)

Akshat Varma (24EARCS015)

Aman Kumar Chauhan (24EARCS017)

Under the guidance of

PROJECT GUIDE

Er. Ram Babu Buri

Department of Computer Science & Engineering

Session: 2026–27

Department of Computer Science & Engineering

ARYA COLLEGE OF ENGINEERING & I.T.

SP-42, RIICO INDUSTRIAL AREA, KUKAS, JAIPUR

CERTIFICATE

This is to certify that the project report entitled "CampusConnect: Student Academic & Communication Portal", submitted by Aditya Gour (24EARCS008), Anjali Kumari (24EARCS020), Aayushman Pathak (24EARCS003), Akshat Varma (24EARCS015), and Aman Kumar Chauhan (24EARCS017) in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in Computer Science & Engineering of Rajasthan Technical University, Kota, is a record of the work carried out by the group at Arya College of Engineering & I.T., Jaipur during the session 2026–27, under the guidance and supervision of Er. Ram Babu Buri.

To the best of our knowledge, the work embodied in this report has not been submitted to any other university or institute for the award of any degree or diploma.

Date: _____ Place: Jaipur

Er. Ram Babu Buri	Dr. Akhil Pandey
Project Guide Dept. of CSE, ACEIT, Jaipur	Head of Department Dept. of CSE, ACEIT, Jaipur

ACKNOWLEDGEMENT

We express our sincere gratitude to our project guide, Er. Ram Babu Buri, for his guidance, encouragement, and valuable suggestions during the planning and development of CampusConnect. His guidance helped us understand the importance of clear requirements, modular design, and systematic verification.

We sincerely thank Dr. Akhil Pandey, Head of the Department of Computer Science and Engineering, Arya College of Engineering & I.T., for the academic environment and support provided for this project. We are also thankful to the faculty members of the department for their cooperation.

We acknowledge the efforts of all team members who contributed to planning, implementation, integration, and review. Working collaboratively with Git and a shared repository helped us understand the importance of communication, version control, and consistent interfaces between modules.

Finally, we thank our families and friends for their encouragement and support throughout the project.

Aditya Gour (24EARCS008)
Anjali Kumari (24EARCS020)
Aayushman Pathak (24EARCS003)
Akshat Varma (24EARCS015)
Aman Kumar Chauhan (24EARCS017)
Department of Computer Science & Engineering

ABSTRACT

CampusConnect is a proposed Student Academic and Communication Portal designed to provide students and teachers with a common web-based platform for accessing institution-related information. The project is being developed initially for Arya College of Engineering & I.T., Jaipur, with a longer-term vision of supporting multiple colleges through institution-specific data and access controls.

The current development focus includes registration and login APIs, student and teacher roles, password hashing using BCrypt, JWT-based authentication, frontend API integration through Axios, and a dashboard flow after successful login. The application uses React on the frontend, Java with Spring Boot on the backend, and MySQL for persistent storage. Spring Security provides the foundation for protecting backend routes and validating authentication.

The report describes the problem context, requirements, technology stack, layered architecture, authentication flow, conceptual data model, module design, security considerations, testing approach, development risks, and future scope. Features such as announcements, assignments, chat, club management, and complete multi-college administration are treated as planned work unless verified in the current implementation. The report does not claim unmeasured performance results or production readiness.

TABLE OF CONTENTS

CHAPTER 1 INTRODUCTION
1.1 Background of the Project
1.2 About the Institution
1.3 Problem Statement
1.4 Objectives of the Project
1.5 Scope of the Project
1.6 Significance and Expected Benefits
1.7 Team Organisation and Working Method
1.8 Organisation of the Report
CHAPTER 2 TECHNOLOGIES AND CONCEPTS
2.1 React.js
2.2 Tailwind CSS and Frontend Tooling
2.3 Java and Spring Boot
2.4 REST APIs and Axios
2.5 MySQL and Relational Data
2.6 Spring Security and JWT
2.7 Password Hashing with BCrypt
2.8 Maven and Application Configuration
2.9 Git and Collaborative Development
2.10 Technology Stack Summary
CHAPTER 3 REQUIREMENT ANALYSIS
3.1 Existing System and Its Limitations
3.2 Proposed System
3.3 User Classes and Roles
3.4 Functional Requirements
3.5 Non-Functional Requirements
3.6 Design Constraints
3.7 Success Criteria
CHAPTER 4 SYSTEM DESIGN
4.1 Architectural Style
4.2 Layered Architecture
4.3 Authentication Request Flow
4.4 Database Design
4.5 Conceptual Entities
4.6 API Design
4.7 Role-Based Access Control
4.8 Multi-College Readiness
CHAPTER 5 MODULE DESIGN AND IMPLEMENTATION APPROACH
5.1 Identity and Authentication Module
5.2 User Profile Module
5.3 Dashboard Module

5.4 API Client and Session Handling
5.5 Database Connectivity and Configuration
5.6 Frontend-Backend Integration
5.7 Planned Academic and Communication Modules
5.8 Suggested Implementation Sequence
CHAPTER 6 SECURITY ARCHITECTURE
6.1 Security Philosophy
6.2 Password Security
6.3 JWT Validation and Session Handling
6.4 Role and Institution Isolation
6.5 CORS, Secrets and Error Handling
6.6 Threats and Mitigations
6.7 Privacy and Data Protection
6.8 Residual Risks
CHAPTER 7 DEVELOPMENT PLAN, TESTING AND EVALUATION
7.1 Development Roadmap
7.2 Testing Strategy
7.3 Representative Test Cases
7.4 Current Status and Evaluation
7.5 Risks and Responses
7.6 Ethical and Operational Considerations
CHAPTER 8 CONCLUSION AND FUTURE SCOPE
8.1 Conclusion
8.2 Limitations
8.3 Future Scope
REFERENCES

CHAPTER 1: INTRODUCTION

1.1 Background of the Project

Colleges use a variety of separate channels to share academic information, communicate with students, distribute assignments, publish announcements, and coordinate activities. When information is scattered across messaging groups, email, paper notices, and independent tools, students may miss updates and faculty members may need to repeat the same communication. CampusConnect is proposed as a unified Student Academic and Communication Portal that brings important campus workflows into one web-based platform.

1.2 About the Institution

The project is being developed for Arya College of Engineering & I.T., Jaipur, Rajasthan, affiliated with Rajasthan Technical University, Kota. The first release is intended to focus on the needs of this institution. The long-term direction is to make the platform capable of supporting additional colleges, with each institution managing its own users, academic information, announcements, and dashboards.

1.3 Problem Statement

Academic communication and student coordination can become fragmented when separate tools are used for authentication, academic notices, assignments, profiles, and student activities. A central portal can improve discoverability, reduce duplication, and provide a consistent access point. At the same time, student and teacher accounts must be protected, and access to information must follow role-based permissions.

1.4 Objectives of the Project

- To develop a web-based portal for students and teachers.
- To provide registration and login using a secure backend authentication flow.
- To distinguish STUDENT and TEACHER roles and support role-aware access.
- To maintain user profile information and present relevant dashboard content.
- To connect the React frontend to Java Spring Boot REST APIs.
- To persist application data in MySQL.
- To plan for institution-specific data and future multi-college expansion.
- To follow a modular structure that supports further academic and communication features.

1.5 Scope of the Project

The current implementation focus is authentication, role selection, profile-related functionality, and dashboards. The wider product vision includes assignments, announcements, chat, club management, and academic sections. These wider features are to be added in phases and must not be treated as completed unless verified in the running application. Multi-college support is a design direction; institution isolation must be implemented and tested before the platform is used by multiple colleges.

1.6 Significance and Expected Benefits

- Students: a single place to sign in and access relevant campus information.
- Teachers: a structured portal foundation for communicating academic information.
- College administrators: a possible central place to manage institutional information as the system grows.
- Development team: practical experience with Java web technologies, REST APIs, authentication, database connectivity, and collaborative software development.

1.7 Team Organisation and Working Method

The project is being developed by a team of five students as an Advanced Java Lab project. Work is divided by modules and coordinated through a shared Git repository. The authentication, user profile, and dashboard work

is maintained on the branch aman-auth-profile. Team member names and university roll numbers should be entered exactly as supplied by the group before final submission.

1.8 Organisation of the Report

Chapter 2 introduces the technologies and concepts. Chapter 3 analyses the requirements. Chapter 4 presents the system architecture and data design. Chapter 5 describes the principal modules and implementation approach. Chapter 6 discusses security. Chapter 7 presents testing, verification and project risks. Chapter 8 concludes the report and identifies future work.

CHAPTER 2: TECHNOLOGIES AND CONCEPTS

2.1 React.js

React is a JavaScript library used to build component-based user interfaces. CampusConnect uses React for its frontend screens and client-side interaction. Components can be reused across authentication pages, dashboards, and future module screens.

2.2 Tailwind CSS and Frontend Tooling

Tailwind CSS supports utility-based styling and responsive layouts. The frontend is developed in a modern JavaScript toolchain and communicates with the backend through HTTP requests. Styling and interface details should remain consistent across student and teacher experiences.

2.3 Java and Spring Boot

Java is the primary backend language for CampusConnect. Spring Boot provides application configuration, REST controllers, dependency injection, service classes, and integration with Spring Security. This structure separates HTTP request handling from business logic and persistence.

2.4 REST APIs and Axios

REST endpoints expose backend operations to the frontend using HTTP methods and JSON payloads. Axios is used for frontend API communication. The shared API client attaches a bearer token when available and handles unauthorized responses by clearing the client token and redirecting to the login screen.

2.5 MySQL and Relational Data

MySQL is the selected relational database. A relational model is suitable for structured information such as users, roles, profiles, departments, academic records, announcements, and assignments. Table relationships, constraints, and indexes should be chosen to maintain data consistency as the project expands.

2.6 Spring Security and JWT

Spring Security provides the security filter chain and authentication integration. JSON Web Tokens (JWTs) can carry signed claims used to identify an authenticated user. CampusConnect's backend includes JWT-related services and a request filter. Protected endpoints must validate tokens and enforce the required permissions on the server.

2.7 Password Hashing with BCrypt

BCrypt is used to hash passwords before storage. A password hash is not the original password and should not be reversible. Credentials must never be written to logs or committed to source control. Database credentials and JWT secrets should be supplied through environment configuration rather than hard-coded into a public repository.

2.8 Maven and Application Configuration

Maven manages backend dependencies and build tasks. The backend is started from the backend directory using Maven. Spring configuration includes the datasource URL, database username and password, driver, JPA settings, server port, CORS settings, and JWT configuration. Environment-specific values must be checked before deployment.

2.9 Git and Collaborative Development

Git supports parallel work through branches, commits, and pull requests. The project team uses a shared repository and a feature branch named aman-auth-profile for the authentication/profile work. Before merging, contributors should pull current changes, inspect conflicts, run the application, and verify the module they changed.

2.10 Technology Stack Summary

Layer	Technology	Purpose
Frontend	React.js	User interface and component rendering
Styling	Tailwind CSS	Responsive interface styling
Backend	Java, Spring Boot	REST APIs and business logic
Security	Spring Security, JWT, BCrypt	Authentication and access control
Database	MySQL	Persistent relational data
API client	Axios	HTTP communication with backend
Build	Maven	Dependency management and backend execution
Version control	Git and GitHub	Team collaboration and change tracking

CHAPTER 3: REQUIREMENT ANALYSIS

3.1 Existing System and Its Limitations

When academic information is distributed among different channels, students may need to search several locations for updates. Faculty may maintain overlapping lists, and account access may not be connected to a consistent portal. A unified system can centralise access while preserving the different responsibilities of students and teachers.

3.2 Proposed System

CampusConnect proposes a browser-based portal with secure authentication, role-aware access, user profiles, and dashboards. The backend validates user input, verifies credentials, and returns the response required by the frontend. Data is persisted through the configured database. The system is planned to expand into announcements, assignments, chat, club management, and academic information.

3.3 User Classes and Roles

Role	Primary responsibility	Access principle
STUDENT	Use student-facing portal features and manage permitted profile fields	Access only permitted student data and operations
TEACHER	Use teacher-facing dashboard and future academic workflows	Access teacher functions and authorised academic data
COLLEGE ADMIN (future)	Manage users and institution-level settings	Restricted to the assigned institution

PLATFORM ADMIN (future)	Manage institution onboarding and platform-level configuration	Cross-institution access only where explicitly authorised

3.4 Functional Requirements

- FR-1: The system shall provide a registration flow with required-field validation.
- FR-2: The system shall allow a registered user to log in with valid credentials.
- FR-3: Passwords shall be stored as secure hashes rather than plaintext.
- FR-4: The backend shall issue and validate authentication tokens according to the configured JWT implementation.
- FR-5: The system shall distinguish student and teacher roles.
- FR-6: Protected APIs shall reject unauthenticated requests.
- FR-7: The frontend shall store and attach the authentication token according to the implemented client flow.
- FR-8: The system shall provide a dashboard after successful authentication.
- FR-9: User profile data shall be validated and persisted when the relevant profile APIs are implemented.
- FR-10: The architecture shall allow future modules to be added without mixing UI, business logic, and persistence responsibilities.
- FR-11: Future multi-college support shall associate users and institution-owned records with an institution identifier.
- FR-12: The backend shall enforce institution and role access rules for all protected operations.

3.5 Non-Functional Requirements

Category	**Requirement**
Security	Password hashing, token validation, server-side authorisation, secret management
Usability	Clear forms, helpful validation messages, responsive screens
Maintainability	Separate controller, service, repository, and configuration responsibilities
Reliability	Useful error responses and stable database connectivity
Performance	Efficient queries and suitable indexes as data volume grows
Scalability	Architecture that can evolve toward institution-scoped data
Portability	Documented local setup for Java, Maven, MySQL, and frontend tooling

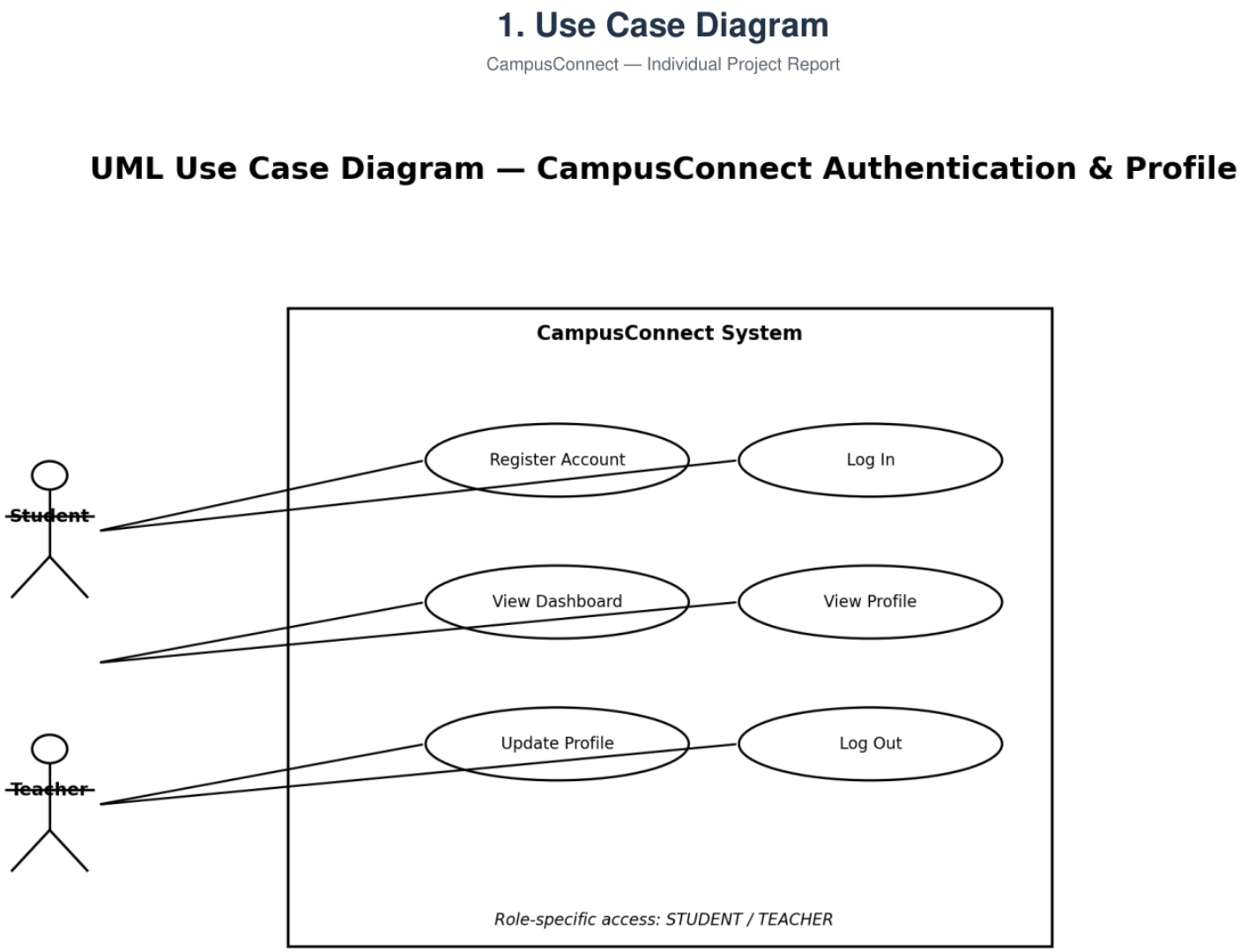
3.6 Design Constraints

- The project is developed within the scope and timeline of an academic PBL.
- The initial deployment target is one college.
- Database credentials and JWT secrets must be configured per environment.
- Features must be marked complete only after execution and testing.
- Multi-college data isolation requires explicit database and service-layer enforcement.

3.7 Success Criteria

The initial success criteria are successful registration and login, correct role assignment, access-token validation, protected-route behaviour, persistence of the relevant user records, stable database connectivity, and a usable dashboard. Quantitative performance and adoption figures are not claimed in this report because no measured results were provided.

Figure 3.1 — UML Use Case Diagram



Supports Section 3.3 by showing the primary STUDENT and TEACHER interactions with CampusConnect.

CHAPTER 4: SYSTEM DESIGN

4.1 Architectural Style

CampusConnect follows a layered web application approach. The React frontend presents screens and calls REST APIs. Spring Boot controllers receive requests and delegate to services. Security filters validate authentication, and persistence components communicate with MySQL. This separation improves testability and makes it easier to add modules in stages.

4.2 Layered Architecture

Layer	Responsibilities
Presentation layer	React pages, forms, dashboard components, client-side validation
API communication	Axios client, JSON requests, bearer-token attachment, response handling
Security layer	Spring Security filter chain, JWT authentication filter, password encoder
Controller layer	REST endpoints for registration, login, and protected operations
Service layer	Validation, business rules, role-specific registration logic
Persistence layer	Data access and entity persistence
Database layer	MySQL tables and relational constraints

4.3 Authentication Request Flow

A user submits the login form. The frontend sends the credentials to the authentication API. The backend validates the request and checks the stored password hash. If authentication succeeds, the server returns the configured token response. The frontend stores the token according to the current implementation and navigates to the dashboard. Subsequent API requests attach the bearer token. The backend validates it before allowing protected access. If a request returns 401 Unauthorized, the frontend clears the token and redirects the user to login.

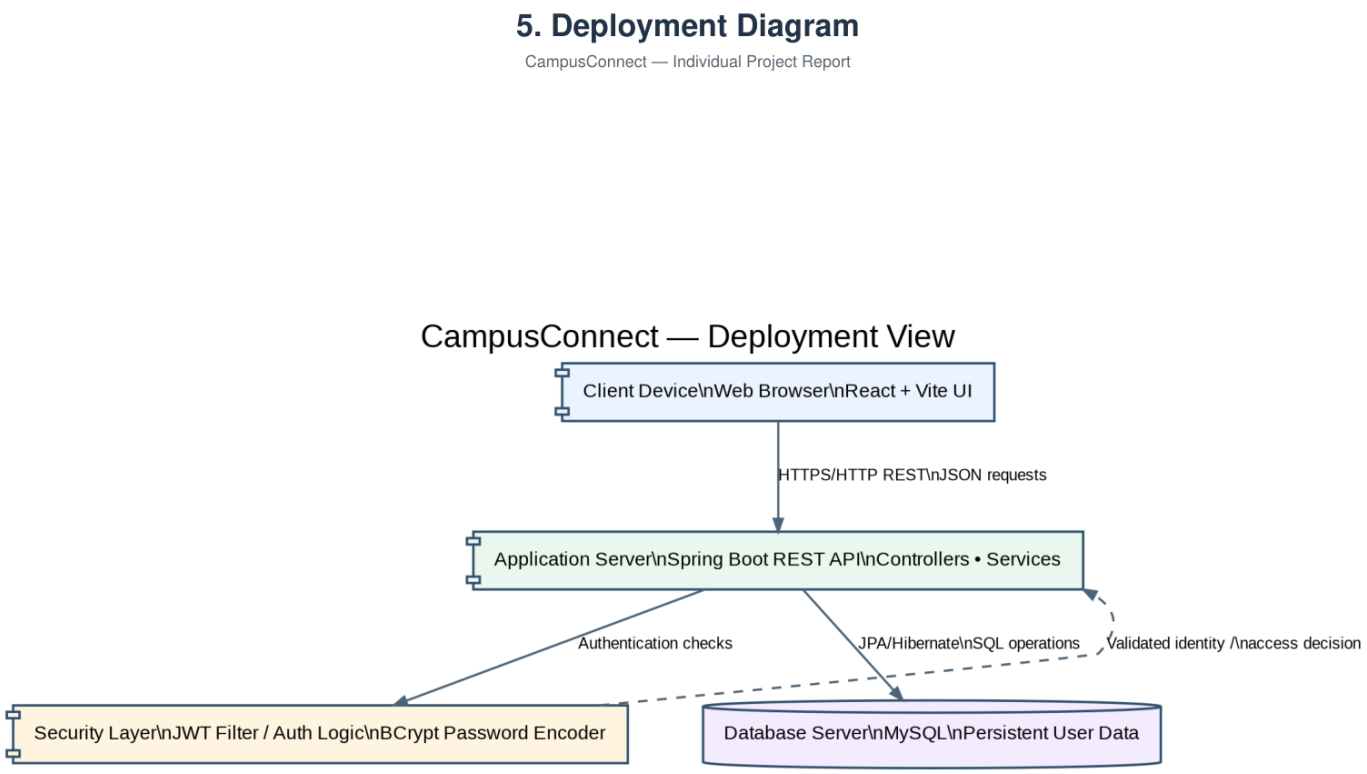
4.4 Database Design

The confirmed database technology is MySQL, and the application configuration uses a database named campusconnect. The exact entity schema should be taken from the current repository before final submission. At a conceptual level, the application needs user identity, role, profile information, and institution association. Future academic features may add assignments, announcements, messages, clubs, and related records.

4.5 Conceptual Entities

Entity	Example attributes	Purpose
User	id, name, email, password_hash, role	Identity and authentication
Student Profile	user_id, enrollment_no, department, semester	Student-specific information
Teacher Profile	user_id, employee_id, department	Teacher-specific information
Institution (future)	id, name, code, status	Institution-specific ownership and configuration
Announcement (planned)	id, institution_id, title, body, created_by, created_at	College communication
Assignment (planned)	id, institution_id, subject_id, title,	Academic task distribution

Figure 4.1 — UML Deployment Diagram

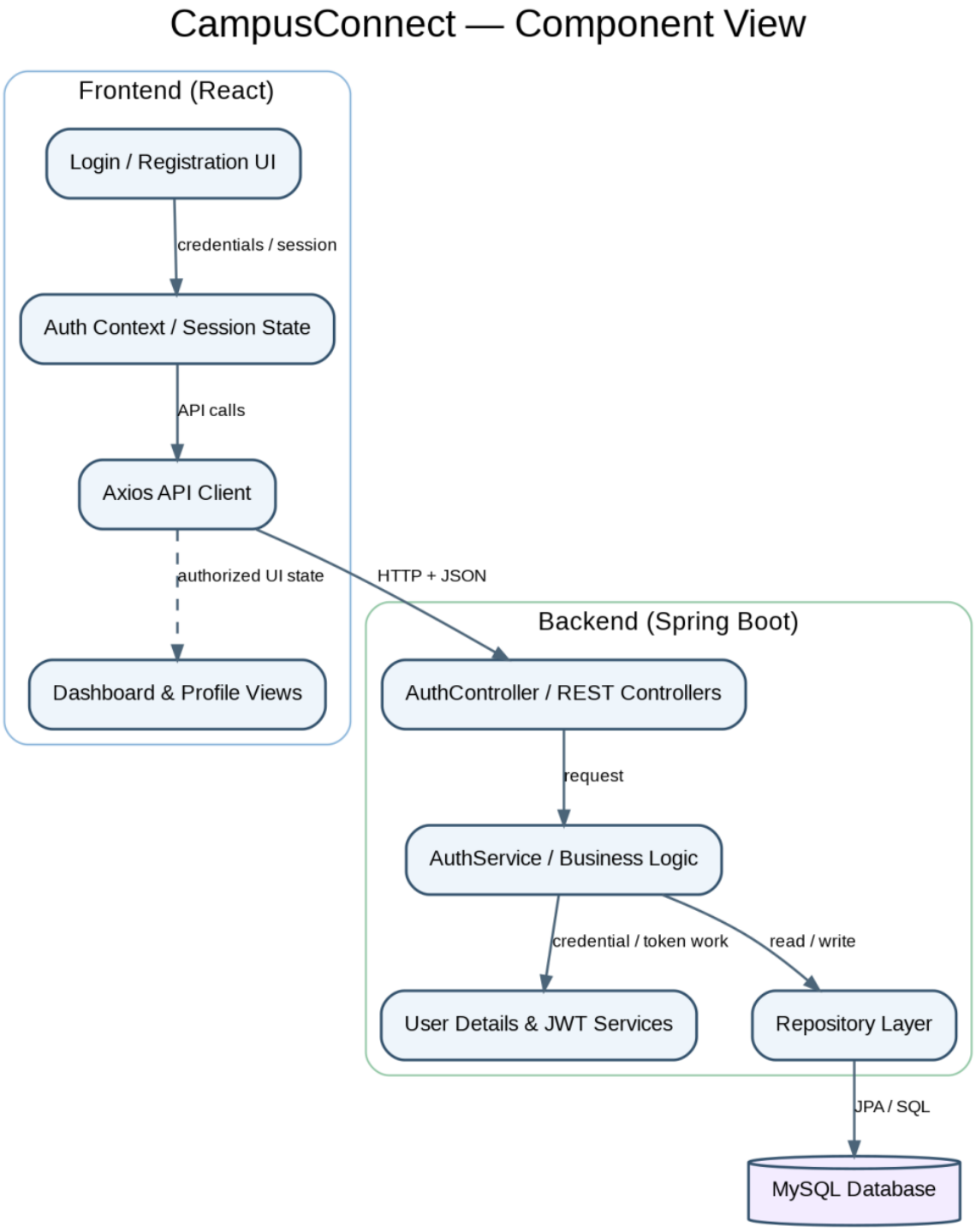


Illustrates the client, Spring Boot application server, security layer, and MySQL database described in Sections 4.1–4.2.

Figure 4.2 — UML Component Diagram

6. Component Diagram

CampusConnect — Individual Project Report



Shows the frontend, authentication/session components, backend services, repository layer, and MySQL integration.

	due_date	

4.6 API Design

The implemented authentication controller exposes registration and login endpoints under /api/auth/register and /api/auth/login. Request and response field names must be confirmed against the current code before they are copied into final API documentation. Other endpoints should follow consistent HTTP status codes, input validation, and structured error handling.

4.7 Role-Based Access Control

Role-based access control assigns permissions according to the authenticated user's role. The current design includes STUDENT and TEACHER. The backend must enforce role restrictions even if the frontend hides a feature. Public registration must not allow a client to grant itself elevated privileges by submitting an arbitrary role. Institution-scoped authorisation is required before supporting multiple colleges.

4.8 Multi-College Readiness

The product vision is to launch at Arya College of Engineering & I.T. first and later support more colleges. A future institution model should associate each user with an institution and scope institution-owned records through that relationship. Every query and mutation must verify the user's institution context. A platform administrator may have a separate, carefully restricted role for onboarding institutions. These are architectural requirements for future implementation, not a claim that multi-tenant isolation is already complete.

CHAPTER 5: MODULE DESIGN AND IMPLEMENTATION APPROACH

5.1 Identity and Authentication Module

The authentication module supports registration and login through Spring Boot REST endpoints. The service validates required information, checks duplicate email addresses, and applies role-specific identifiers such as enrollment number for students and employee ID for teachers. Passwords are encoded using BCrypt, and the JWT service creates tokens for authenticated sessions. The module should be tested for invalid credentials, duplicate users, malformed requests, and attempts to submit an unauthorised role.

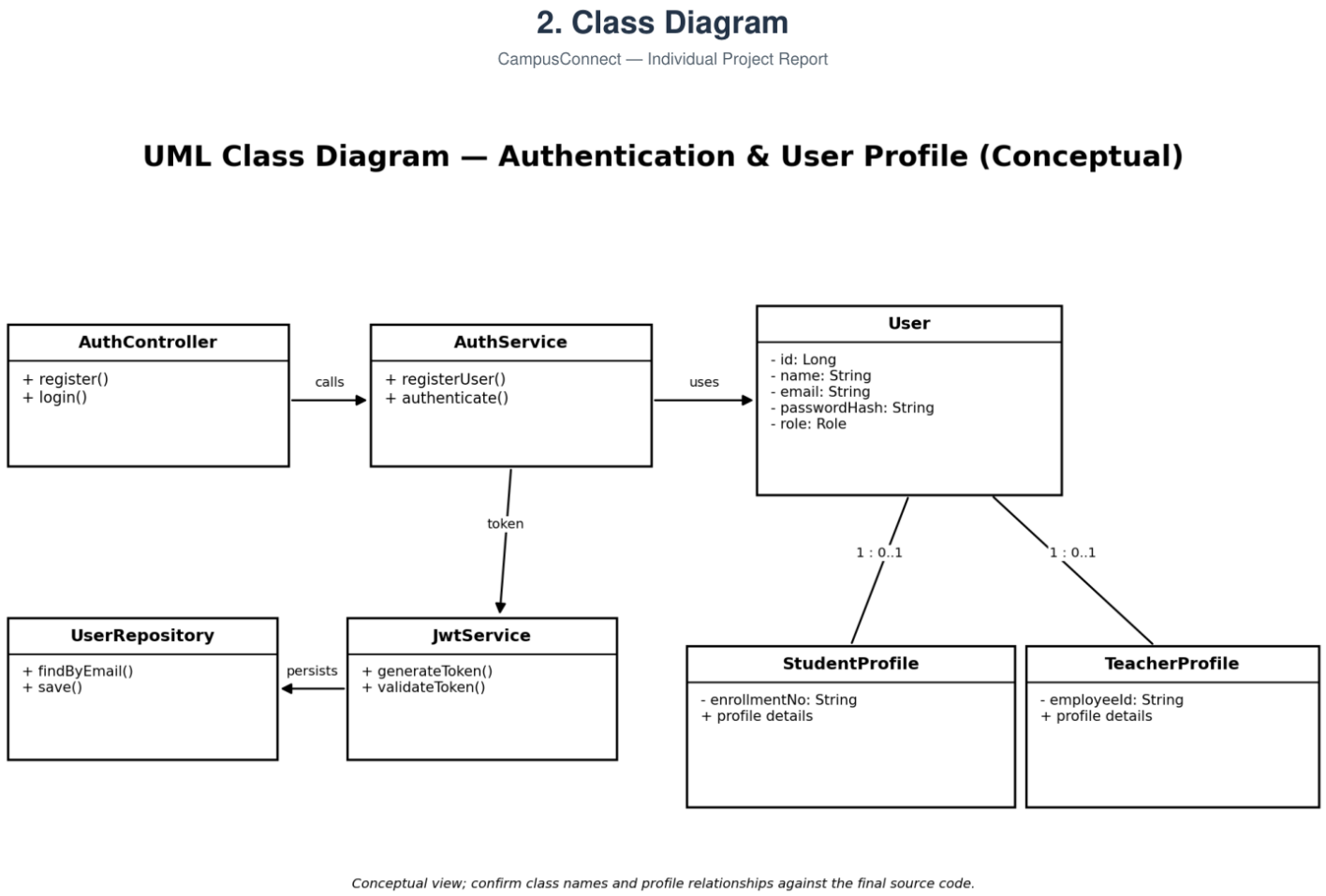
5.2 User Profile Module

The profile module is intended to store and display user information associated with the authenticated account. Student and teacher profiles may require different fields. Profile updates should validate permitted fields and derive the account identity from the authenticated principal rather than trusting a user-supplied identifier. The current repository should be reviewed to confirm which profile read and update operations are complete.

5.3 Dashboard Module

After successful login, the user is directed to a dashboard. The dashboard is the entry point for role-relevant information. The initial version should clearly distinguish the student and teacher experience and avoid displaying data that the current user is not authorised to view. Future dashboard widgets can display announcements, assignment deadlines, messages, club activities, and academic summaries as those modules become available.

Figure 4.3 — UML Class Diagram

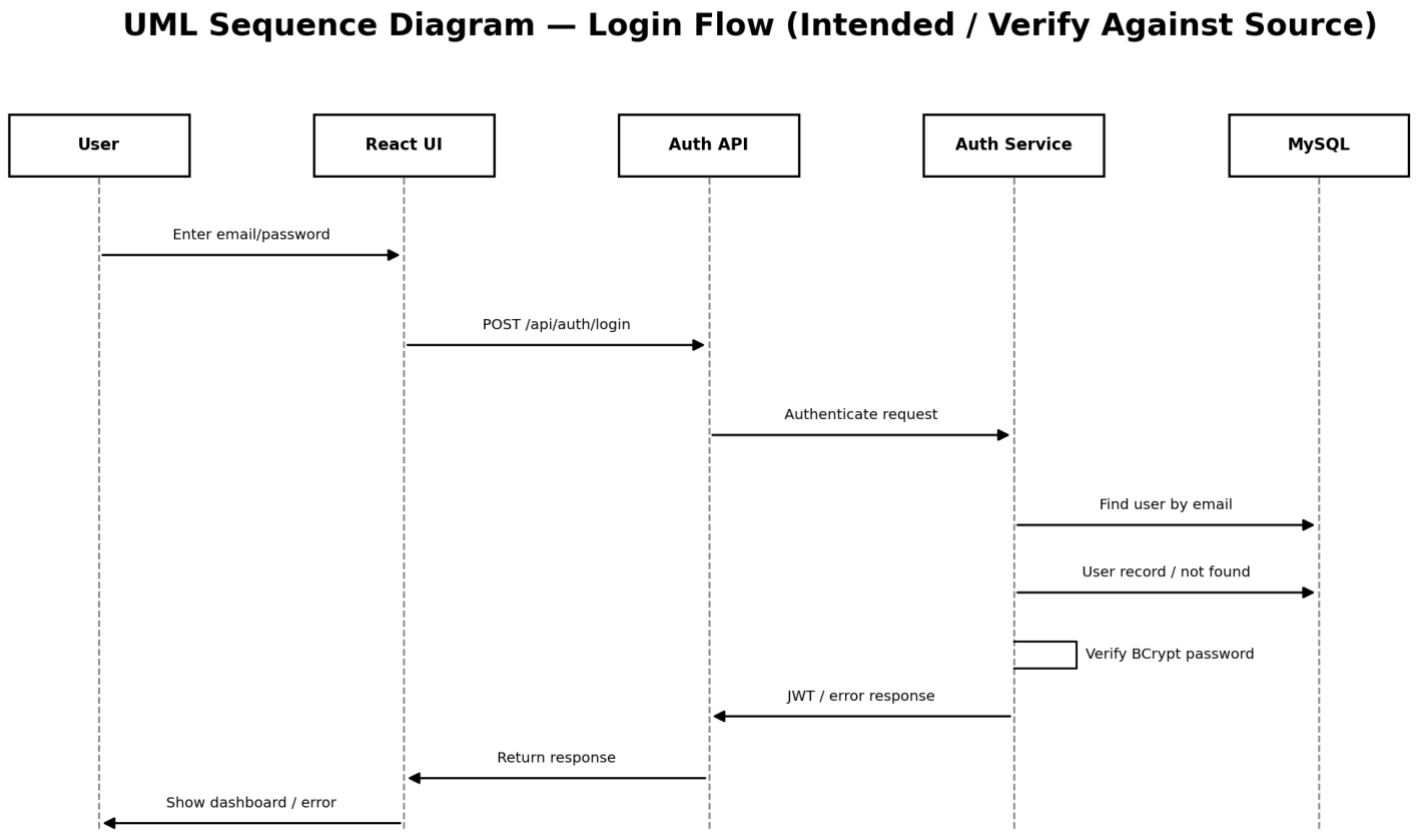


Presents the conceptual relationships among authentication controllers/services, user data, JWT services, repositories, and role-specific profiles.

Figure 4.4 — UML Login Sequence Diagram

3. Login Sequence Diagram

CampusConnect — Individual Project Report

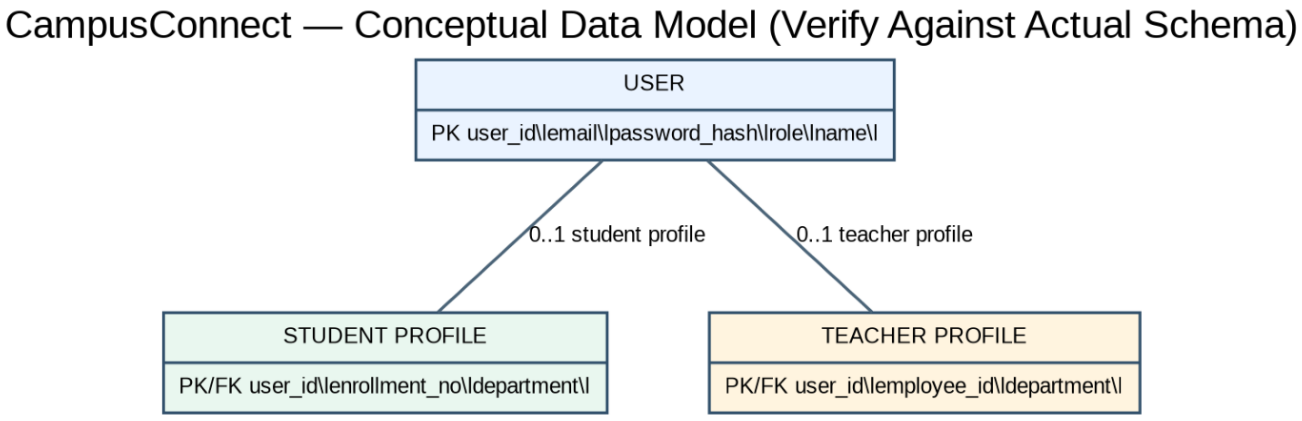


Visualises the authentication request flow described in Section 4.3.

Figure 4.5 — ER Diagram

7. ER Diagram

CampusConnect — Individual Project Report



Provides the conceptual data model referenced in Sections 4.4–4.5.

5.4 API Client and Session Handling

The frontend API client uses Axios interceptors to attach a bearer token from localStorage under the key cc_token. The response interceptor handles 401 responses by clearing the token and redirecting to /login. This is the current implementation approach recorded during development. Before production deployment, token storage and cross-site scripting risks should be reviewed; HttpOnly cookies may be preferable depending on the final session design.

5.5 Database Connectivity and Configuration

The backend uses Spring datasource configuration for a local MySQL server and the campusconnect database. The configuration includes a JDBC URL, database username and password, the MySQL driver, JPA settings, CORS allowed origins, and JWT settings. Actual credentials should be provided through local environment variables or an untracked environment-specific configuration file. They should never be included in the report or committed to Git.

5.6 Frontend-Backend Integration

The frontend sends HTTP requests to the backend API. Successful authentication should result in a token response that matches the frontend's expected response structure. If the backend changes field names, status codes, or token format, the API client and AuthContext must be updated together. Integration tests should verify both the network response and the resulting frontend navigation.

5.7 Planned Academic and Communication Modules

- Announcements: institution-wide or role-targeted notices with timestamps and author information.
- Assignments: teacher-created assignments, due dates, attachment support, and student submissions.
- Chat: authenticated communication with appropriate membership and privacy controls.
- Club management: club profiles, activity notices, and membership workflows.
- Academic section: structured information by department, semester, subject, and academic year.

5.8 Suggested Implementation Sequence

First stabilise registration, login, token validation, and database persistence. Next verify profile access and dashboard behaviour for both roles. Then add institution ownership and access tests before enabling multi-college operation. After that, implement announcements and assignments as complete vertical slices, followed by chat and club management. This sequence reduces the risk of building feature screens before the identity and authorisation foundations are dependable.

CHAPTER 6: SECURITY ARCHITECTURE

6.1 Security Philosophy

CampusConnect handles account credentials and may eventually handle academic records and private communications. Security therefore needs to be enforced at the backend and database boundaries, not only in the user interface. Every protected operation should identify the caller, verify the token, check the required authority, validate input, and return only the data that caller is allowed to access.

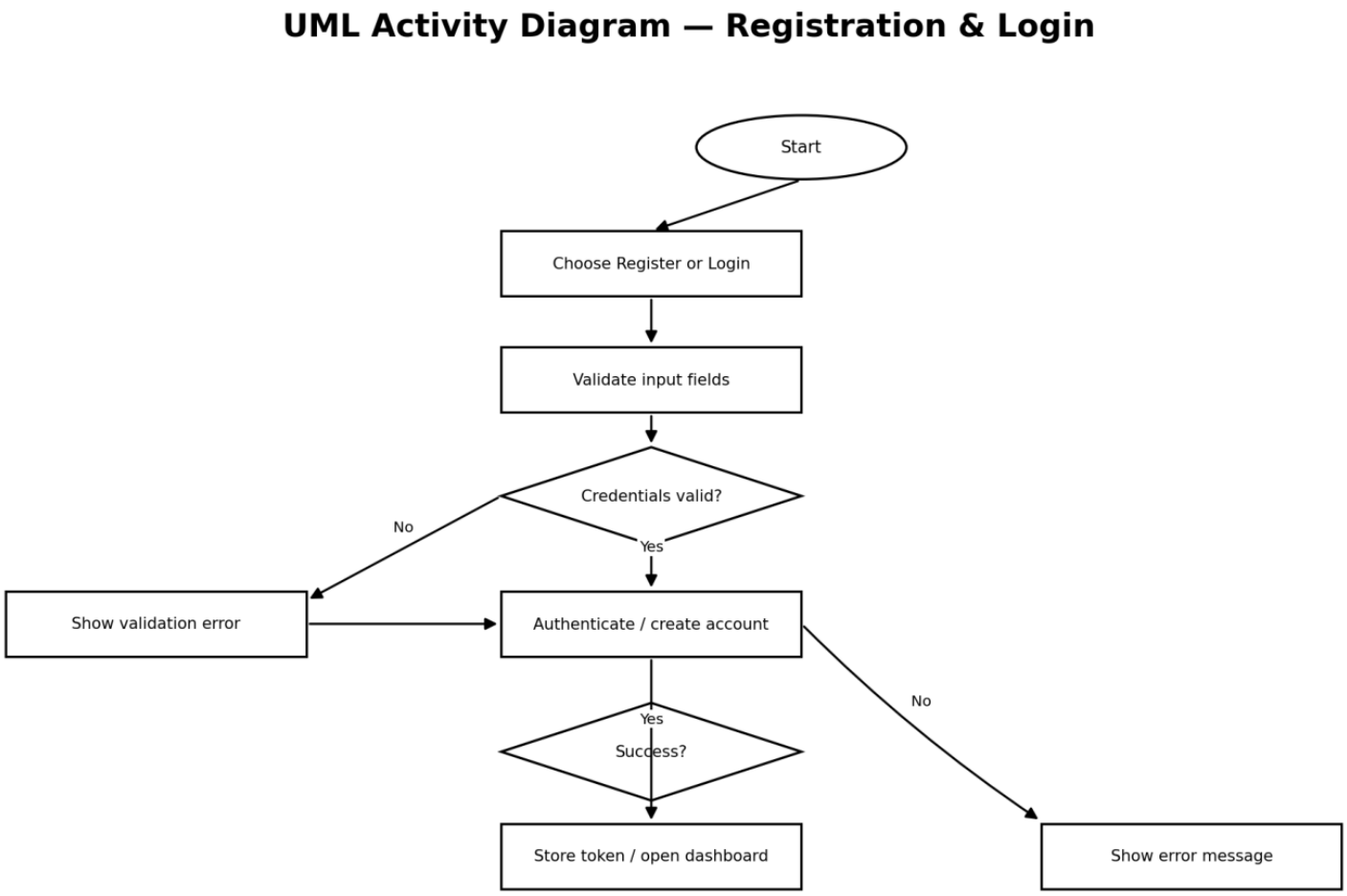
6.2 Password Security

Passwords should be encoded with BCrypt and never stored or logged in plaintext. The backend should use a suitable work factor, validate password requirements consistently, and avoid exposing whether a password or account detail was the exact cause of an authentication failure where that would aid account enumeration.

Figure 5.1 — UML Activity Diagram

4. Activity Diagram

CampusConnect — Individual Project Report



Shows the registration and login activity flow supporting the Identity and Authentication Module.

6.3 JWT Validation and Session Handling

JWT validation must verify the signature, expiration, and expected claims. Signing secrets must be sufficiently strong and supplied through environment configuration. Logout and token expiry behaviour should be verified against the current implementation. If refresh tokens are added, they should be protected, rotated, and revocable according to the selected session design.

6.4 Role and Institution Isolation

A user's role must be assigned and checked by trusted backend code. Public registration must not allow a user to become a teacher or administrator simply by editing the request payload. For multi-college operation, every institution-owned record must be scoped to the correct institution, and a user from College A must not be able to retrieve or modify College B's data by changing an identifier in a request.

6.5 CORS, Secrets and Error Handling

CORS should allow only the frontend origins required by the deployment. Database passwords and JWT secrets must be excluded from source control. Error responses should provide enough information to troubleshoot expected failures without revealing secrets, stack traces, or sensitive database details to ordinary users.

6.6 Threats and Mitigations

Threat	Potential issue	Mitigation
Credential exposure	Plaintext or leaked passwords	BCrypt; do not log credentials; protect configuration
Token theft	Unauthorised use of a valid token	HTTPS, short token lifetime, careful storage, expiry checks
Privilege escalation	Client submits TEACHER or admin role	Server-controlled role assignment and authorisation
Cross-college data exposure	Changing IDs to access another institution's records	Institution scoping in services and database queries
SQL injection	Untrusted values affect database queries	Parameterized repository queries and input validation
Misconfiguration	Secrets or permissive CORS committed to Git	Environment variables, secret review, restricted origins

6.7 Privacy and Data Protection

The portal should collect only the information required for academic and communication workflows. Access to student profiles and future messages should be limited to authorised purposes. Data retention, account recovery, audit logging, and deletion requirements should be defined before production deployment. The report does not claim formal legal compliance; compliance obligations should be assessed with the institution before launch.

6.8 Residual Risks

The implementation still requires verification of token storage, logout behaviour, all protected endpoint rules, institution scoping, deployment configuration, and backup/recovery practices. These items should remain in the project checklist until tested in the running system.

CHAPTER 7: DEVELOPMENT PLAN, TESTING AND EVALUATION

7.1 Development Roadmap

Phase	Deliverable	Verification
1	Backend and frontend setup	Application starts with documented commands
2	Registration and login	Valid credentials succeed; invalid credentials fail
3	JWT security integration	Protected APIs reject missing or invalid tokens
4	Student and teacher roles	Role-specific rules are enforced by backend
5	Profiles and dashboard	Correct profile and dashboard shown for each role
6	Institution foundation	Users and data are scoped to one institution
7	Announcements and assignments	Create/read/update operations pass role checks
8	Chat and clubs	Membership and visibility rules are verified
9	Integration and regression tests	Core journeys work after merges
10	Deployment and documentation	Environment setup, backup, and release steps documented

7.2 Testing Strategy

Testing should include unit tests for service validation, integration tests for MySQL persistence and authentication, API tests for HTTP status codes and role permissions, and end-to-end checks of registration, login, dashboard navigation, and logout. Tests should be repeated after merging teammates' changes because shared controllers, security configuration, and frontend contexts can affect several modules.

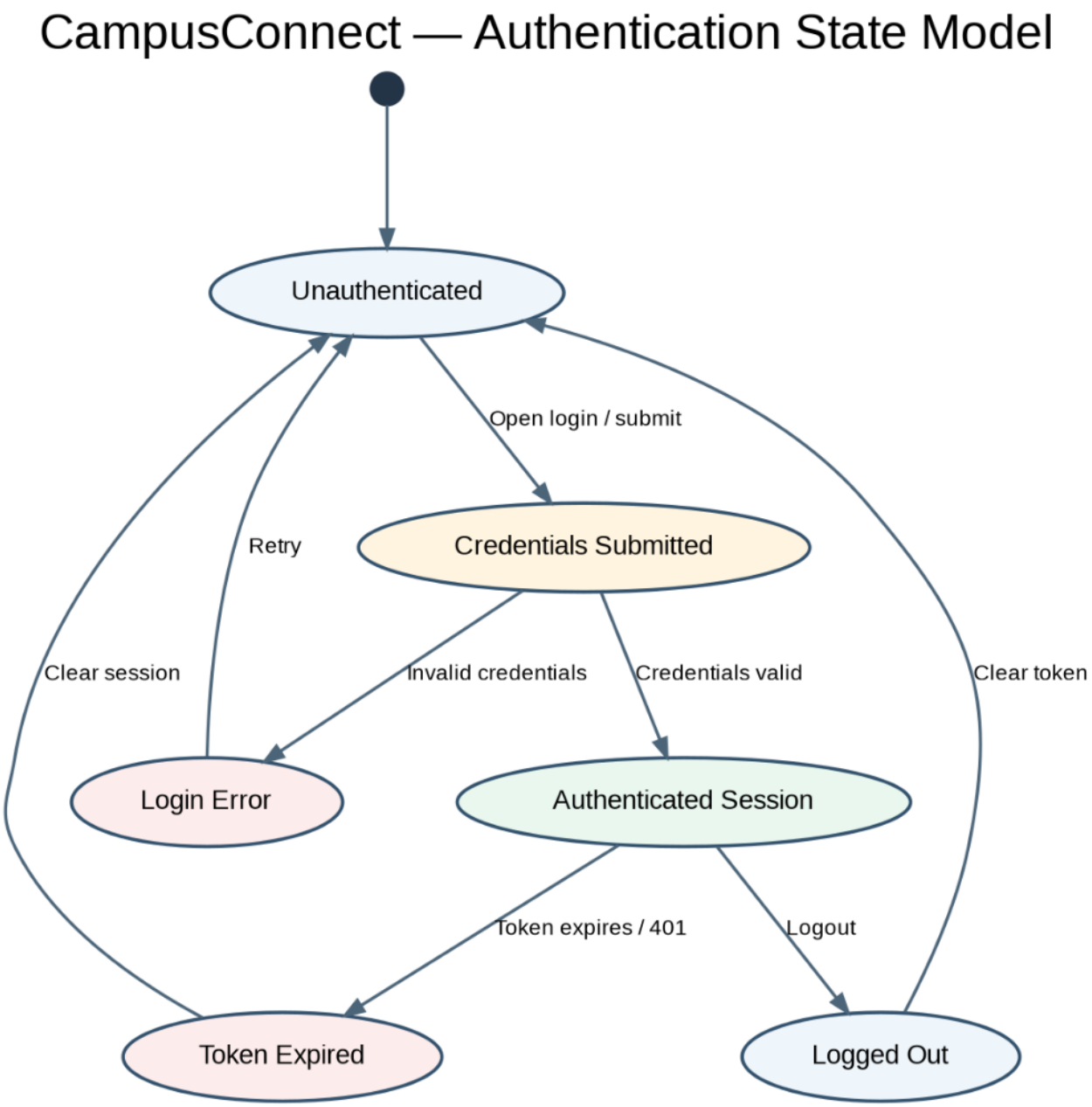
7.3 Representative Test Cases

ID	Scenario	Expected result
T1	Register with valid required fields	Account is created and password is hashed
T2	Register with an already-used email	Request is rejected with a clear validation response
T3	Login with valid credentials	Authentication succeeds and expected token response is returned
T4	Login with invalid password	Authentication fails without exposing sensitive details
T5	Call protected API without a token	Request is rejected
T6	Call protected API with expired/invalid token	Request is rejected
T7	Student attempts teacher-only operation	Access is denied by backend
T8	Submit elevated role in public registration	Client cannot grant itself extra privileges
T9	Restart backend and query persisted account	Persisted data remains available
T10	User from one institution requests another's data (future)	Access is denied
T11	Wrong database credentials	Application reports a

Figure 6.1 — Authentication State Diagram

8. Authentication State Diagram

CampusConnect — Individual Project Report



Shows the authentication states and transitions associated with login, token expiry, errors, and logout.

		configuration/connectivity error safely
T12	Frontend receives 401 response	Token is cleared and user is redirected to login

7.4 Current Status and Evaluation

Development notes indicate that the authentication endpoints, JWT-related security components, BCrypt encoding, Axios token handling, and login-to-dashboard flow have been implemented. The backend previously encountered a Maven startup/build failure and database configuration required setup. Consequently, the system should be described as under development until a clean build, repeatable startup, persistence checks, and role-based API tests pass. No benchmark figures or production-readiness claims are made here.

7.5 Risks and Responses

Risk	Impact	Response
Database configuration error	Backend fails to start or connect	Validate database name, service status, credentials, and JDBC URL
Merge conflict	Shared code becomes inconsistent	Pull latest changes, review diffs, run tests before merging
Token mismatch	Login succeeds but later API calls fail	Document response contract and test Axios/AuthContext together
Feature scope growth	Core authentication remains unstable	Finish and test vertical slices before adding modules
Cross-institution leakage	Serious privacy and security issue	Implement institution scoping and negative access tests before onboarding colleges

7.6 Ethical and Operational Considerations

Academic information should be accurate, access should be limited to legitimate users, and students should receive clear feedback when an operation fails. Administrative overrides, if introduced, should be authorised and auditable. Before launch, the college should decide ownership of records, support responsibilities, backup schedules, and how account corrections are handled.

CHAPTER 8: CONCLUSION AND FUTURE SCOPE

8.1 Conclusion

CampusConnect is a Student Academic and Communication Portal intended to provide a common digital entry point for students and teachers. Its current technical foundation combines a React frontend with a Java Spring Boot backend, MySQL persistence, JWT-based authentication, Spring Security, and BCrypt password hashing. The authentication and dashboard work establishes a foundation for future academic and communication modules.

8.2 Limitations

- The project is still under development and requires repeatable build and runtime verification.
- The current scope focuses on authentication, roles, profiles, and dashboards; wider modules are planned.
- The exact database schema and every API contract should be checked against the current repository before final submission.

- Multi-college isolation is a future architectural requirement and must be implemented and tested before multi-college use.
- No production performance, adoption, or security-audit results are claimed.

8.3 Future Scope

- Complete and verify profile editing for students and teachers.
- Add a college administration area for managing institution-level settings.
- Implement institution-scoped users and records, followed by automated cross-college isolation tests.
- Introduce announcements and assignment workflows.
- Add secure messaging and club management.
- Improve notifications, search, accessibility, and responsive design.
- Add automated unit, integration, API, and end-to-end tests to CI.
- Deploy using managed configuration, HTTPS, database backups, monitoring, and a documented recovery process.

REFERENCES

Technical References

[1] React Documentation, https://react.dev/

[2] Spring Boot Reference Documentation, https://docs.spring.io/spring-boot/

[3] Spring Security Reference, https://docs.spring.io/spring-security/reference/

[4] MySQL Documentation, https://dev.mysql.com/doc/

[5] Apache Maven Documentation, https://maven.apache.org/guides/

[6] JSON Web Token (JWT), RFC 7519, https://www.rfc-editor.org/rfc/rfc7519

[7] BCrypt password hashing, OpenBSD manual pages and implementation documentation.

[8] Axios Documentation, https://axios-http.com/

[9] Tailwind CSS Documentation, https://tailwindcss.com/docs

[10] Git Documentation, https://git-scm.com/doc

[11] CampusConnect project repository and implementation notes, consulted for the current project-specific architecture and status.

APPENDIX A: DETAILED USER REQUIREMENTS

Supplementary specification for the first college-focused release

A.1 Purpose and boundaries

This appendix converts the project objectives into practical requirements that can guide implementation and review. It distinguishes the first release from the broader CampusConnect vision. The initial deployment target is Arya College of Engineering & I.T., Jaipur. Features that are not yet implemented are described as requirements or planned capabilities, not as completed functionality.

A.2 Student user stories

ID	User story	Acceptance condition
ST-01	As a student, I want to register using my required student identifier so that my account can be associated with my academic identity.	Required fields are validated; duplicate email/identifier is rejected; valid data is persisted.
ST-02	As a student, I want to log in with my credentials so that I can access my account.	Valid credentials succeed; invalid credentials do not create a session.
ST-03	As a student, I want to view my profile so that I can check the information associated with my account.	Only the authenticated user or an authorised administrator can access protected profile data.
ST-04	As a student, I want a dashboard that presents relevant campus information.	The dashboard loads without exposing teacher-only or other users' private data.
ST-05	As a student, I want to see announcements and assignments in a later release.	Items are scoped to the correct college, academic audience, and publication status.

A.3 Teacher user stories

ID	User story	Acceptance condition
TE-01	As a teacher, I want to register with an employee identifier where required.	The identifier is validated and unique within the agreed scope.
TE-02	As a teacher, I want to log in and reach a role-aware dashboard.	The backend derives the role from trusted account data and the UI reflects permitted functions.
TE-03	As a teacher, I want to publish academic notices in a future module.	Only authorised teachers can create or edit notices for their permitted audience.
TE-04	As a teacher, I want to create	Required fields and dates are

	assignments with due dates in a future module.	validated; students see only relevant assignments.

A.4 Administrator and institution requirements

- An institution record should identify each participating college and its approved configuration.
- Administrative functions should be restricted to explicitly authorised accounts; a student or teacher role must not automatically grant administrative privileges.
- Institution-specific records should carry a trusted institution reference, with enforcement in service and repository queries.
- Administrative actions that change roles, publish notices, or remove records should be auditable.
- Before onboarding another college, cross-institution access tests and operational ownership must be defined.

A.5 Requirement priority

Priority	Meaning	Examples
P0 — Essential	Required for a usable first release.	Reliable build, registration/login, role assignment, database persistence, protected access.
P1 — Important	Needed to make the portal useful beyond a demonstration.	Profile viewing/editing, role-aware dashboard, clear validation and error messages.
P2 — Planned	Adds academic workflows after the foundation is stable.	Announcements, assignments, file attachments, academic calendar.
P3 — Future	Requires additional design and operational work.	Real-time chat, analytics, mobile application, multi-college administration.

APPENDIX B: SYSTEM FLOWS AND DESIGN VIEWS

Logical views of the current architecture and intended request behaviour

B.1 Context view

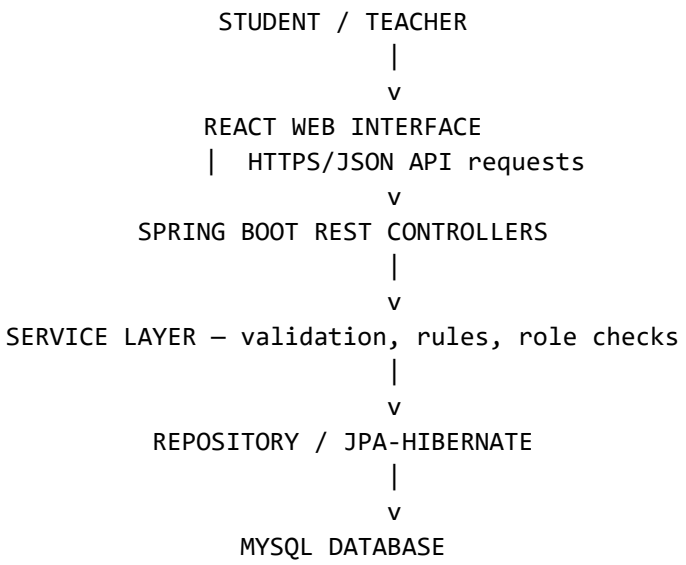


Figure B.1. Logical context view. The diagram shows the intended separation of responsibilities; the exact classes and mappings should be verified against the current repository.

B.2 Registration flow

1. The user opens the registration screen and selects the appropriate account category.
2. The frontend performs basic field checks to improve usability; these checks do not replace backend validation.
3. The frontend sends the registration payload to the authentication endpoint.
4. The backend validates required fields and checks uniqueness constraints such as email and the role-specific identifier.
5. The password is encoded using the configured password encoder before it is stored.
6. The user record is committed to MySQL only if the transaction completes successfully.
7. The API returns a response that does not disclose the password or password hash.
8. The frontend displays a clear success or validation message and allows the user to proceed to login.

B.3 Login and authenticated request flow

9. The user submits the login form over the configured API connection.
10. The backend loads the account and verifies the submitted password against the stored encoded value.
11. On successful authentication, the token service issues a signed token according to the configured claims and expiration.
12. The frontend stores the token using the current client-side session strategy and attaches it as a Bearer token on subsequent API calls.
13. The backend filter validates the token before a protected request is accepted.
14. Service-layer authorization checks whether the authenticated user may perform the requested operation on the target resource.
15. The API returns only the data permitted for that account; authentication or authorization failures return an appropriate error status.

B.4 Logical module relationships

Layer	Responsibility	Should not do
React components	Render forms, navigation, dashboards, validation feedback.	Hold database credentials or make access-control decisions.
API client	Centralise base URL, token attachment, response handling.	Treat a client-side role label as authoritative.
Controller	Map HTTP requests to application operations and responses.	Contain complex persistence logic.
Service	Apply business rules, validation, transactions, and authorization checks.	Expose password hashes or trust client-supplied ownership.
Repository / ORM	Load and persist entities using defined mappings.	Decide user-facing navigation or UI behaviour.
MySQL	Persist structured application records with integrity constraints.	Be reachable directly from the public browser.

APPENDIX C: CONCEPTUAL DATA DICTIONARY

Logical entities to validate against the actual JPA entity classes

The following is a conceptual data dictionary, not a claim that every table already exists. Before final implementation, align entity names, column names, nullability, indexes, and relationships with the current database migrations or JPA mappings.

C.1 User account

Field concept	Purpose	Validation / note
User ID	Stable primary identifier for an account.	Use a database-generated or otherwise unique identifier.
Email	Login/contact identifier.	Normalise consistently and enforce the intended uniqueness scope.
Password hash	Encoded password representation.	Never store or return plaintext passwords.
Role	Distinguishes STUDENT and TEACHER.	Use an allowed server-side enum/value; never accept arbitrary elevated roles.
Status	Allows active/disabled account handling if implemented.	Disabled accounts should not authenticate.
Created / updated time	Supports audit and troubleshooting.	Use a consistent time-zone policy.

C.2 Student profile and teacher profile

Entity	Possible fields	Design consideration
Student profile	Enrollment number, name, department, semester, batch, contact preference.	Collect only required fields; protect student identifiers from unnecessary disclosure.
Teacher profile	Employee ID, name, department, designation, contact preference.	Employee identifiers should be validated and exposed only where necessary.
Institution	Institution ID, name, campus code, active status.	Needed before real multi-college operation; users and records must be scoped to it.
Department	Department ID, institution ID, department name.	Avoid ambiguous department names across colleges.
Announcement (planned)	Title, body, author, audience, publish time, status, institution ID.	Check both author permissions and audience/institution scope.
Assignment (planned)	Title, description, due date, creator, course/section,	Validate due dates and restrict file type/size if attachments are

	attachments metadata.	enabled.

C.3 Relationships and integrity

- A user account should map to the profile type appropriate to its role, with consistency enforced in backend logic and, where feasible, database constraints.
- Role-specific identifiers should be unique within the agreed scope. If the system later supports several colleges, uniqueness may need to be defined per institution rather than globally.
- An institution-owned record should reference its institution explicitly instead of relying on a name supplied by the browser.
- Records such as assignments and announcements should reference their author and intended audience so access rules can be evaluated consistently.
- Deletion policy should be chosen deliberately: academic records may require archival or soft deletion instead of immediate physical deletion.

C.4 Data retention and quality

The project should define which fields are mandatory, who can correct them, and how long records are retained. Input validation should trim unintended whitespace, enforce length limits, reject malformed values, and use consistent handling for email addresses. The team should avoid collecting sensitive data that is not necessary for the portal's academic purpose.

APPENDIX D: API CONTRACT AND ERROR-HANDLING GUIDELINES

Illustrative contract checklist; exact request and response fields must match the source code

D.1 Authentication endpoints

Endpoint	Purpose	Validation points
POST /api/auth/register	Create a user account.	Required fields, email format, duplicate email, role-specific identifier, password policy, permitted role values.
POST /api/auth/login	Authenticate an existing account.	Required credentials, password verification, account status, safe error response, token expiry.

Request and response JSON examples are intentionally not hard-coded here because the exact DTO field names and response shape must be taken from the current implementation. The report should not document a field that the application does not actually accept or return.

D.2 HTTP status conventions

Status	Typical meaning	Client behaviour
200 OK	Request succeeded, commonly login or data retrieval.	Process the response and update the view.
201 Created	A resource was created, if used by the endpoint.	Show success and use returned safe resource metadata.
400 Bad Request	Malformed input or validation failure.	Show field-level feedback where available.
401 Unauthorized	Credentials are invalid or authentication is missing/expired.	Clear invalid client session state and ask the user to sign in.
403 Forbidden	Authenticated user lacks permission.	Do not retry with the same permission; show an access message.
404 Not Found	Resource or route does not exist or is intentionally hidden.	Show a safe not-found state.
409 Conflict	A uniqueness or state conflict, if mapped this way.	Explain the conflict without leaking private account details.
500 Internal Server Error	Unexpected server failure.	Show a generic message; use server logs for diagnosis.

D.3 Error response principles

- Keep response formats consistent across endpoints so the frontend can handle errors predictably.
- Do not return stack traces, SQL statements, password hashes, signing secrets, or internal filesystem paths to the browser.

- Validation messages should be useful but must not reveal whether a sensitive account exists when that information could be abused.
- Log operational details on the server with request correlation information, while avoiding credentials and full tokens.
- Test both success and failure responses, including malformed JSON, duplicate registration, invalid passwords, expired tokens, and forbidden role actions.

D.4 Frontend session behaviour

The frontend API client is described in the project notes as using Axios interceptors and the localStorage key cc_token. The exact implementation should be checked before submission. A 401 response should remove stale client-side credentials and route the user to login without creating a redirect loop. UI route guards are useful for navigation but do not replace backend authorization.

APPENDIX E: TEST PLAN AND TRACEABILITY

A repeatable test checklist for demonstration and future regression testing

E.1 Test levels

Level	Focus	Evidence to retain
Unit	Service validation, role logic, token utility behaviour.	Test names, assertions, pass/fail output.
Integration	Controller-service-repository interaction and MySQL persistence.	Test configuration and database cleanup strategy.
API	HTTP status codes, payload validation, authentication headers.	Request/response records with secrets removed.
UI	Form validation, loading states, success/error messages, route navigation.	Screenshots and reproducible steps.
End-to-end	Registration → login → dashboard → logout.	Browser and backend versions, result, known limitations.
Security	Invalid token, expired token, role tampering, cross-user access attempts.	Expected denial and actual status/result.

E.2 Representative test cases

Test ID	Scenario	Expected result	Actual result
TC-01	Register with valid student details.	Account is created once; safe response is returned.	To be recorded during execution.
TC-02	Register with an existing email.	Request is rejected without creating a duplicate.	To be recorded during execution.
TC-03	Register with missing required fields.	Validation error is returned.	To be recorded during execution.
TC-04	Login with correct credentials.	Authentication succeeds and token response matches the frontend contract.	To be recorded during execution.
TC-05	Login with incorrect password.	Authentication fails; no token is issued.	To be recorded during execution.
TC-06	Call a protected endpoint without a token.	Request is rejected.	To be recorded during execution.
TC-07	Call a protected endpoint with an invalid/expired	Request is rejected.	To be recorded during execution.

	token.		
TC-08	Student attempts teacher-only operation.	Backend denies the action.	To be recorded when that operation exists.
TC-09	Restart the backend and log in again.	Persisted account can be authenticated if the database is available.	To be recorded during execution.
TC-10	Stop MySQL and start the backend.	Startup failure is diagnosable; no secret values are printed in public responses.	To be recorded during execution.

E.3 Requirement traceability

Requirement group	Related tests	Evidence
Registration and uniqueness	TC-01, TC-02, TC-03	API response and database verification.
Login and password handling	TC-04, TC-05	Login result; no plaintext credential storage.
Token validation	TC-06, TC-07	Protected endpoint response.
Role-based access	TC-08	Denied request from the wrong role.
Persistence and configuration	TC-09, TC-10	Restart test and controlled startup logs.
Profile/dashboard	UI and end-to-end checks	Role-specific screens and permitted profile data.

Important: do not fill the Actual result column with 'Pass' unless the team has run the test and recorded the outcome. This report uses 'To be recorded during execution' to distinguish the planned test plan from verified results.

APPENDIX F: LOCAL SETUP AND DEMONSTRATION GUIDE

For development and academic demonstration only

F.1 Prerequisites

- Java Development Kit compatible with the backend configuration; the repository README should be treated as the final source for the required version.
- Apache Maven installed and available in the terminal.
- MySQL Server running, with a database and least-privilege database account configured for the application.
- Node.js and npm installed for the React/Vite frontend.
- Git and a code editor for repository management.

F.2 Configuration checklist

16. Open the project root and review the README and backend configuration before starting services.
17. Create the MySQL database required by the configured datasource. Confirm the database name and schema-generation/migration approach used by the project.
18. Supply the datasource username and password through the configuration mechanism used by the application. Do not commit real credentials to Git.
19. Supply a strong JWT signing secret through the intended environment configuration if token signing is enabled.
20. Confirm the frontend API base URL matches the backend address and port.
21. Check the configured CORS origins against the local frontend origin; do not use permissive production settings.

F.3 Typical development commands

```
Backend (from the backend directory):
  mvn spring-boot:run

Frontend (from the frontend directory):
  npm install
  npm run dev
```

These are typical commands documented for the repository's technology stack. Use the project README and actual folder names if they differ. The backend and frontend run as separate processes, so keep both terminals available while testing.

F.4 Demonstration sequence

22. Start MySQL and verify the target database is reachable.
23. Start the backend and wait for a successful application startup message.
24. Start the frontend and open the local URL printed by Vite.
25. Register a test student account with non-sensitive sample data.
26. Log in and confirm the dashboard is displayed.
27. Check the browser console and backend terminal for errors, without sharing tokens or passwords.
28. Log out, then test invalid credentials and protected-route behaviour.
29. Capture screenshots of the actual running application for the final report.

F.5 Troubleshooting guide

Symptom	Likely checks

Backend exits during startup	Read the first meaningful exception above the final Maven failure summary; verify Java version, missing environment variables, and port conflicts.
Database connection refused	Confirm MySQL service is running, host/port are correct, and credentials are valid.
Unknown database	Create the configured database or correct the datasource database name.
Login returns 401	Check credentials, password encoding, account record, token configuration, and the frontend/backend response contract.
Browser blocks API request	Check backend availability, frontend API URL, and allowed CORS origin.
Dashboard appears but refresh loses session	Check token persistence, session hydration, and 401 interceptor behaviour.
Maven command not found	Verify Maven installation and PATH, then reopen the terminal.

APPENDIX G: USER INTERFACE AND ACCESSIBILITY GUIDELINES

Quality principles for a student-facing portal

G.1 Consistency and navigation

A portal should make its main sections predictable. The header, navigation labels, page titles, and action buttons should use consistent terminology. Students and teachers should be able to identify their current page and return to the dashboard without relying only on the browser back button.

G.2 Forms and feedback

- Label every field clearly and indicate required fields in text, not only by colour or an asterisk.
- Keep entered values when validation fails, except where security requires clearing them.
- Show progress during network requests and prevent accidental duplicate submissions.
- Use a clear success message after a completed operation and an actionable message when it fails.
- Do not expose stack traces or raw server exceptions in the interface.
- Ensure password fields support show/hide controls only when implemented accessibly and do not log input values.

G.3 Responsive behaviour

The interface should remain usable on common laptop and mobile viewport sizes. Layouts should avoid horizontal scrolling for forms and primary navigation. Tables containing academic information may need a responsive card view or a deliberate horizontal-scroll region. The team should test at least one desktop viewport and one narrow mobile viewport before recording screenshots.

G.4 Accessibility checklist

Area	Checklist
Keyboard	All interactive elements can be reached and activated using a keyboard.
Focus	Focus is visible and follows a logical sequence.
Labels	Inputs have programmatic labels; placeholders are not the only labels.
Contrast	Text and controls have sufficient contrast against their backgrounds.
Errors	Validation errors are associated with the relevant field and described in text.
Semantics	Headings follow a meaningful hierarchy; buttons and links are used for their intended purpose.
Motion	Avoid unnecessary animation; respect reduced-motion preferences where applicable.

G.5 Interface evaluation record

For each screenshot included in the final submission, record the screen name, viewport size, test account role, date captured, and whether the shown data is sample data. Use only genuine screenshots from the current application. Do not use mock-ups as evidence that a feature is implemented.

APPENDIX H: PROJECT MANAGEMENT AND TEAM COORDINATION

A lightweight workflow for a five-member academic project

H.1 Work breakdown

Workstream	Typical tasks	Completion evidence
Authentication and profile	Registration/login, validation, role-specific identifiers, profile access.	Successful and failure-path tests; code review.
Frontend integration	Forms, API client, token handling, navigation, responsive states.	Screenshots and browser test notes.
Database and configuration	Schema review, datasource setup, local environment instructions.	Repeatable setup on a clean environment.
Academic modules	Announcements, assignments, clubs, chat, academic information.	Requirements, data model, endpoints, and tests before marking complete.
Quality and documentation	Integration testing, diagrams, report, setup guide, limitations.	Reviewed report and reproducible demonstration.

H.2 Git collaboration practices

30. Create a focused feature branch for each coherent change rather than committing unrelated modules together.
31. Pull the latest shared branch before beginning integration work.
32. Commit small, descriptive changes and avoid including generated build folders, local credentials, or IDE-specific noise.
33. Open a pull request or request peer review for changes affecting shared interfaces, database models, or security.
34. Resolve merge conflicts deliberately; after merging, build and test the integrated application rather than assuming each branch will work together.
35. Record any API contract change so frontend and backend contributors remain aligned.

H.3 Definition of done

- The change meets an agreed requirement and has no known critical regression.
- Input validation and failure handling have been considered.
- Relevant tests have been run and their outcomes recorded.
- Configuration changes are documented without publishing secrets.
- Code is committed to the intended branch and reviewed where practical.
- Documentation and screenshots match the actual current behaviour.

H.4 Suggested milestone review

Milestone	Review question

M1 — Build foundation	Can a new developer install dependencies and start the application using the instructions?
M2 — Authentication	Can valid users register and log in, and are invalid requests rejected safely?
M3 — Profile and dashboard	Does each role see only the information and actions intended for it?
M4 — Academic modules	Are announcements/assignments scoped correctly and validated at the backend?
M5 — Release candidate	Do integration tests pass, are known limitations documented, and can the demo be repeated?

APPENDIX I: GLOSSARY AND ABBREVIATIONS

Key terms used in this report

Term	Meaning in this project
API	Application Programming Interface; the contract used by the frontend to communicate with backend operations.
BCrypt	A password-hashing algorithm designed for storing password verifiers rather than recoverable plaintext passwords.
CORS	Cross-Origin Resource Sharing; browser-enforced rules that control which origins may call a server from a web page.
CRUD	Create, Read, Update, Delete operations on application data.
DTO	Data Transfer Object; a structured request or response representation used at an API boundary.
HTTP	Hypertext Transfer Protocol, used for request/response communication.
JPA	Java Persistence API, a standard for mapping Java objects to relational data.
JWT	JSON Web Token; a signed token format often used to carry authentication claims.
ORM	Object-Relational Mapping; a technique for mapping program objects to relational database structures.
RBAC	Role-Based Access Control; permission decisions based on roles assigned to authenticated accounts.
REST	An architectural style commonly used to expose resources and operations through HTTP APIs.
SPA	Single-Page Application; a web application that updates views in the browser without a full document navigation for every action.
SQL	Structured Query Language, used to define and manipulate relational database data.
TLS/HTTPS	Transport security used to protect data in transit between client and server.
UI	User Interface; the screens and controls through which users interact with the application.
Vite	A frontend development/build tool used by the React application.

XSS	Cross-Site Scripting; a class of vulnerability in which attacker-controlled script executes in a user's browser.

I.1 Acronyms used for roles

Label	Meaning
STUDENT	A student account intended to access student-facing information.
TEACHER	A teacher/faculty account intended to access faculty-facing workflows.
ADMIN (future/if implemented)	A privileged institutional management role; it must not be inferred from student or teacher status.

APPENDIX J: REVIEW CHECKLIST BEFORE FINAL SUBMISSION

Complete this checklist against the running application and source repository

J.1 Technical verification

- Confirm the backend compiles successfully from a clean terminal.
- Confirm MySQL starts and the application connects to the configured database.
- Confirm the frontend starts and can reach the backend API.
- Verify registration, duplicate handling, valid login, and invalid login.
- Verify whether JWT signing and validation are enabled in the current branch; align the report with the actual code.
- Verify whether Spring Security is actively enforcing protected routes or is only present as a dependency/configuration plan.
- Verify the actual entity classes and schema before presenting a physical ER diagram.
- Verify profile read/update behaviour for both STUDENT and TEACHER roles.
- Run the test cases that will be demonstrated and record actual outcomes.

J.2 Report consistency

- Ensure every team member's name and enrollment number match the official college record.
- Check the guide, HOD, college name, university, session, and department details on the cover and certificate.
- Update the table of contents and page numbers after final PDF export.
- Add genuine screenshots of the login, registration, dashboard, and profile screens if available.
- Label conceptual diagrams as conceptual and planned modules as future scope.
- Remove any claim of measured performance, completed testing, or production readiness unless evidence exists.
- Open the exported PDF and inspect the cover, page breaks, tables, headers, and final references.

J.3 Evidence log

Evidence item	Status / file location
Backend build output	To be recorded by the team.
Registration and login test	To be recorded by the team.
Database persistence check	To be recorded by the team.
Student dashboard screenshot	Insert genuine screenshot if available.
Teacher dashboard screenshot	Insert genuine screenshot if available.
Repository commit/branch used for submission	Record the exact branch and commit hash.

This appendix is intended as a final quality gate. It is better to report a clearly stated limitation than to claim a feature is complete without running it.